



Universidad Nacional Experimental de Guayana.

Vicerrectorado Académico

Proyecto de carrera: Ingeniería en informática.

Asignatura: Lenguajes y Compiladores.

Análisis Léxico

Profesor:

Felix Marquez

Alumnos:

S1. Palma Franyibeth CI 31.564 .549.

S1 .Rodriguez Edfrank CI 30.857.264

S2. Jesus Baez CI 26.753.871

S2.Javier Useche CI 28.700.959

Ciudad Guayana ,julio 2026.

Introducción

En el vasto campo de la teoría de la computación y el diseño de compiladores, la transición de los modelos de reconocimiento teórico a la implementación de software práctico constituye uno de los pilares fundamentales de la informática moderna. El presente documento aborda de manera integral este espectro, partiendo de los fundamentos de los Autómatas de Pila. A diferencia de los autómatas finitos, cuya capacidad de memoria es estrictamente limitada, estos modelos incorporan una estructura de pila con acceso de tipo el último en entrar es el primero en salir. Esta adición teórica les otorga un salto cualitativo en la Jerarquía de Chomsky, permitiéndoles reconocer Lenguajes Libres de Contexto y resolver problemas complejos de recursividad y anidamiento imposibles de procesar mediante expresiones regulares o autómatas finitos convencionales.

Con el objetivo de aterrizar estos conceptos teóricos a soluciones tecnológicas tangibles, este trabajo documenta el desarrollo y la aplicación práctica de tres aproximaciones distintas a la tokenización y el análisis del lenguaje. En primer lugar, se analiza la simulación algorítmica en Python de un simulador para el autómata de pila encargado de reconocer un lenguaje formal específico de potencias emparejadas. En segundo lugar, se detalla el análisis léxico desde cero para el lenguaje de configuración Dockerfile, aplicando expresiones regulares con grupos con nombre para procesar estructuras complejas, priorizar lexemas y gestionar errores de sintaxis en un entorno de integración continua. Por último, se examina la metacompilación industrial mediante el uso de la herramienta clásica Flex en entornos de sistemas operativos libres para generar de manera automatizada un analizador léxico eficiente en lenguaje C, aplicado primero a un subconjunto simplificado del lenguaje Rust y, posteriormente, extendido al ámbito de la seguridad informática mediante la tokenización de reglas de detección de intrusos para el sistema Snort. A través de este recorrido, se evidencia cómo la abstracción matemática de los autómatas se traduce directamente en las herramientas de software encargadas de traducir, analizar y proteger los sistemas de información actuales.

Autómatas de Pila

-Definición de Autómata de Pila (AP)

Un Autómata de Pila (Pushdown Automaton o PDA por sus siglas en inglés) es un modelo computacional que extiende la capacidad de los Autómatas Finitos (tanto Deterministas como No Deterministas) mediante la incorporación de una memoria adicional estructurada en forma de pila (stack).

Mientras que un autómata finito solo tiene una cantidad finita de memoria en forma de estados, un autómata de pila posee una memoria infinita de tipo LIFO (Last-In, First-Out; el último en entrar es el primero en salir). Esto le permite procesar y recordar secuencias complejas, ya que puede "apilar" (push) símbolos en la memoria y "desapilar" (pop) símbolos cuando sea necesario según las reglas de transición.

Formalmente, un Autómata de Pila se define como una 7-tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, donde:

- * **Q**: Conjunto finito de estados.
- * **Σ (Sigma)**: Alfabeto de entrada (símbolos que puede leer).
- * **Γ (Gamma)**: Alfabeto de la pila (símbolos que puede guardar en la pila).
- * **δ (Delta)**: Función de transición (determina el siguiente estado y la acción en la pila en base al estado actual, el símbolo leído y el símbolo en el tope de la pila).
- * **$q_0 \in Q$** : Estado inicial.
- * **$Z_0 \in \Gamma$** : Símbolo inicial de la pila (usado para saber cuándo la pila está vacía).
- * **$F \subseteq Q$** : Conjunto de estados finales o de aceptación.

-Ejemplos de Autómatas de Pila

Ejemplo 1: El lenguaje $L = \{ a^n b^n \mid n \geq 0 \}$

Este lenguaje consiste en todas las cadenas que tienen un número "n" de letras 'a' seguidas exactamente por el mismo número "n" de letras 'b' (ej: nulo, ab, aabb, aaabbb). Un autómata finito no puede resolver este problema porque no tiene memoria para contar cuántas 'a' han pasado. Un autómata de pila lo resuelve de la siguiente manera:

1. Por cada 'a' que lee en la entrada, apila un símbolo (ej. 'A') en la pila.
2. Cuando empieza a leer las 'b', por cada 'b' desapila una 'A'.
3. Si al terminar de leer la cadena, la pila queda vacía (es decir, solo queda el símbolo inicial), significa que hubo la misma cantidad de 'a' y de 'b', por lo que acepta la cadena. Si sobran 'A' en la pila, o faltan 'A' para emparejar con las 'b', rechaza la cadena.

Ejemplo 2: Verificación de Paréntesis Balanceados

Lenguajes de programación y expresiones matemáticas requieren que los paréntesis estén correctamente cerrados (ej: `'(())()'`, `'(())()'`).

1. El autómata lee la expresión de izquierda a derecha.
2. Al leer un paréntesis de apertura `'('`, lo apila.
3. Al leer un paréntesis de cierre `)'`, desapila un elemento del tope de la pila. Si el tope de la pila no es un `'('`, o si la pila está vacía antes de tiempo, la expresión está desbalanceada (rechazo).
4. Si al terminar de leer la cadena la pila está vacía, los paréntesis estaban perfectamente balanceados.

-Conclusión de las Utilidades del Autómata de Pila

Los autómatas de pila tienen utilidades prácticas fundamentales en el campo de la informática y el desarrollo de software. Su utilidad principal radica en el análisis sintáctico (Parsing).

Son la base teórica de los lenguajes libres de contexto. En la construcción de compiladores e intérpretes, las herramientas que analizan la sintaxis del código fuente (parsers) están implementadas utilizando principios de autómatas de pila. Por ejemplo, se utilizan para evaluar expresiones aritméticas con prioridades de operadores, validar que los bloques de código como `'if'`, `'while'`, o `'{'` estén correctamente estructurados y anidados, y procesar formatos de datos jerárquicos como XML o HTML. Sin la estructura LIFO de una pila, las computadoras no podrían procesar lenguajes con estructuras recursivas y anidadas.

-Importancia Relacionada a su Poder con Relación a los AFD y AFND

La importancia del Autómata de Pila se entiende mejor al compararlo con los Autómatas Finitos Deterministas (AFD) y No Deterministas (AFND).

* **Poder Computacional:** Los Autómatas de Pila son estrictamente más poderosos que los AFD y los AFND.

* **Lenguajes que reconocen:** Mientras que los AFD y AFND solo pueden reconocer Lenguajes Regulares (aquellos que pueden ser descritos mediante expresiones regulares, donde no se requiere memoria para "contar" o emparejar de forma arbitraria), los Autómatas de Pila pueden reconocer Lenguajes Libres de Contexto (LLC).

* **La Limitación de los AFD/AFND:** Un AFD/AFND no puede recordar cantidades arbitrarias de información. Por ejemplo, no puede recordar si vio n cantidades de 'a' para luego exigir n cantidades de 'b', porque requeriría infinitos estados para representar cada valor posible de n .

* **El Salto Cualitativo del AP:** La adición de la pila provee memoria infinita controlada. Esto permite resolver problemas de emparejamiento, recursividad y anidamiento. Todo lenguaje regular puede ser reconocido por un autómata de pila (simplemente ignorando la pila), pero no todo lenguaje libre de contexto puede ser reconocido por un AFD/AFND. Por lo tanto, el AP representa el siguiente nivel en la Jerarquía de Chomsky, siendo indispensable para definir los lenguajes de programación modernos que poseen reglas sintácticas complejas.

-Documentación de la Solución de Software

Como parte de esta actividad, se ha construido un programa que simula un Autómata de Pila básico. A continuación, se detallan los aspectos técnicos de esta solución.

Descripción del Programa

El programa construido (`automata_pila.py`) implementa la clase `AutomataDePila` configurada específicamente para reconocer el lenguaje $L = \{a^n b^n \mid n \geq 0\}$.

Este script simula las transiciones de estado y las operaciones de memoria (apilar y desapilar) carácter por carácter. Incluye un método ``procesar_cadena(cadena)`` que determina si una cadena dada es aceptada o rechazada, imprimiendo en consola el paso a paso del procesamiento, el estado de la pila y el veredicto final.

Tecnología y Herramientas Utilizadas

- * **Lenguaje de Programación:** Python 3.x
- * **Paradigma:** Programación Orientada a Objetos (POO). Se encapsuló la lógica y los datos (estado actual, pila) en una clase.
- * **Herramientas:** Cualquier editor de código (VS Code, Sublime Text, etc.) y la terminal/consola del sistema operativo para la ejecución. No se requirieron librerías de terceros.

Detalle de Instalación

Dado que el programa es un script de Python estándar utilizando únicamente la biblioteca estándar, no requiere de una instalación compleja ni gestión de dependencias externas.

Los pasos previos son:

1. Verificar que se tiene instalado Python 3 en el sistema. Se puede descargar desde la **[página oficial de Python](<https://www.python.org/downloads/>)**.
2. Descargar o clonar el archivo ``automata_pila.py`` del repositorio entregado.

Detalle de Ejecución

Para probar y ejecutar el simulador del autómata de pila, se deben seguir estos pasos:

1. Abra una terminal o línea de comandos (Símbolo del sistema, PowerShell, Git Bash, etc.).
2. Navegue utilizando el comando ``cd`` hasta el directorio donde se encuentra guardado el archivo ``automata_pila.py``.
3. Ejecute el script escribiendo el siguiente comando:

```
```bash
python automata_pila.py
```
```

4. Al ejecutarse, el programa correrá automáticamente el bloque de pruebas definido al final del archivo. Este evaluará una serie de casos de prueba válidos (ej. `"aabb"`) e inválidos (ej. `"abb"`), mostrando por consola cada símbolo leído, la acción sobre la pila y si la cadena fue **ACEPTADA** o **RECHAZADA**.

Construcción de un Lexer para la Verificación de Archivos Docker mediante Expresiones Regulares

Fundamento Teórico Aplicado

De acuerdo con la teoría revisada, todo compilador requiere un mecanismo inicial —el analizador léxico— que lea el programa fuente carácter a carácter y reconozca los componentes léxicos (tokens) definidos para el lenguaje L. Dado que los tokens de un lenguaje de programación son, en la práctica, expresiones regulares, y que los lenguajes regulares son reconocidos por un AFD $(Q, \Sigma, \delta, q_0, F)$, es posible evitar la construcción manual de la tabla de transiciones del autómata y, en su lugar, codificar directamente cada estado de aceptación como un patrón regex con nombre. La unión de todos los patrones mediante alternancia $(|)$ conforma una única expresión regular que actúa como el autómata completo: cada vez que se alcanza un estado de aceptación (un token reconocido) el proceso regresa automáticamente al estado inicial para continuar leyendo, tal como se describe en el material del Tema 4.

Esta actividad corresponde al enfoque "desde cero con regex" (a diferencia del enfoque con metacompilador como Flex, usado en el ejemplo del archivo interfaces de Debian). Se utilizó el lenguaje Python y su módulo estándar `re`, aplicando la misma técnica de grupos con nombre $(?P<TOKEN>patrón)$ y `re.finditer()` presentada en el ejemplo de referencia del material del tema.

Diseño del Lenguaje L (Dockerfile)

Antes de construir el lexer se delimitó el subconjunto léxico del lenguaje Dockerfile que se desea reconocer:

- Instrucciones reservadas: FROM, RUN, CMD, LABEL, MAINTAINER, EXPOSE, ENV, ADD, COPY, ENTRYPOINT, VOLUME, USER, WORKDIR, ARG, ONBUILD, STOPSIGNAL, HEALTHCHECK, SHELL.
- Comentarios que inician con # (se ignoran, no generan token).
- Imágenes base, con o sin tag (imagen:tag) y con o sin digest (imagen@sha256:...).
- Banderas de instrucción de una o dos guiones (-y, --from=builder, --chown=usuario:grupo).
- Asignaciones clave=valor (usadas por ENV y ARG) y expansión de variables (\$VAR).
- Cadenas entre comillas y arreglos en forma exec: ["cmd", "arg1"].
- Rutas absolutas y relativas, números (incluyendo notación de versión 1.0.0), mapeos y números de puerto.
- Operadores de encadenamiento de shell dentro de RUN: &&, |, ;.

La Tabla 1 resume el conjunto final de tokens definidos y su patrón regex asociado (se muestra la forma resumida; el patrón completo se encuentra en el código fuente `lexer_dockerfile.py`).

Tabla 1. Componentes léxicos (tokens) del lexer

| Token | Patrón regex (resumido) | Ejemplo de lexema |
|---------------------|----------------------------------|-----------------------|
| COMMENT | #.* | # comentario |
| INSTRUCTION | ^(FROM RUN CMD ...)\b | FROM, RUN, COPY |
| UNKNOWN_INSTRUCTION | ^[A-Z][A-Z_]{1,}\b | FORM (typo de FROM) |
| FLAG | --?[a-zA-Z][a-zA-Z0-9-]**(=...)? | --from=builder, -y |
| IMAGE_DIGEST | nombre@sha256:[0-9a-f]{64} | alpine@sha256:c158... |
| PORT_MAP | \d{1,5}:\d{1,5}/(tcp udp)? | 8080:80/tcp |
| PORT | \d{1,5}/(tcp udp) | 8080/tcp |
| IMAGE_TAG | nombre:tag | python:3.12-slim |
| ASSIGN | [A-Za-z_][A-Za-z0-9_]*= | PORT=, BUILD_VERSION= |
| VARIABLE | \\$VAR \${VAR} | \$RUTA |

| | | |
|-----------------------------|----------------------------|------------------------|
| STRING_DOUBLE/SINGLE | "..." '...' | "python" |
| AND_OP / PIPE_OP / SEMI | && \ ; | && |
| LBRACKET / RBRACKET / COMMA | [] , | ["a", "b"] |
| DOT_PATH | \.{1,2}(/...)? |/cmd |
| PATH | /[^\t"]* | /usr/local/bin |
| NUMBER | \d+(\.\d+)* | 8080, 1.0.0 |
| IDENTIFIER | [A-Za-z_][A-Za-z0-9_\.\/]* | requirements.txt |
| NEWLINE / SKIP | \n [\t]+ | (se ignoran) |
| MISMATCH | . | carácter no reconocido |

Construcción del Lexer Paso a Paso

El desarrollo siguió los mismos pasos generales descritos en la teoría del Tema 4 para construir un analizador léxico basado en regex, adaptados al lenguaje Dockerfile:

Paso 1 - Delimitar el alfabeto y las palabras reservadas

Se listó el conjunto de instrucciones válidas de un Dockerfile (variable INSTRUCCIONES) y se generó, mediante join, la alternancia regex correspondiente (_INSTR_ALT). Esta lista actúa como el vocabulario reservado del lenguaje L, equivalente al conjunto de estados de aceptación 'palabra clave' del AFD.

Paso 2 - Definir un patrón regex por cada componente léxico

Se creó la lista TOKENS con tuplas (nombre_token, patrón_regex): cada patrón equivale a un estado de aceptación distinto del autómata (COMMENT, INSTRUCTION, FLAG, IMAGE_TAG, IMAGE_DIGEST, PORT, PORT_MAP, ASSIGN, VARIABLE, STRING, LBRACKET/RBRACKET/COMMA, PATH, NUMBER, IDENTIFIER, NEWLINE, SKIP y, finalmente, MISMATCH como estado de error).

Paso 3 - Resolver conflictos de prioridad entre patrones

Al igual que ocurre al construir un AFD a mano, varios patrones pueden coincidir con la misma entrada (por ejemplo, '8080:80/tcp' podría interpretarse como IMAGE_TAG o como PORT_MAP). Este conflicto se resolvió mediante el ORDEN de las alternativas dentro de la expresión regular combinada: Python re.finditer() prueba las alternativas en el orden en que aparecen, por lo que los patrones más específicos (IMAGE_DIGEST, PORT_MAP, PORT) se colocaron antes que los patrones más generales (IMAGE_TAG, IDENTIFIER).

Paso 4 - Combinar todos los patrones en una única expresión regular con grupos con nombre

Se construyó TOKEN_REGEX uniendo todas las tuplas con el patrón (?P<NOMBRE>patrón) separadas por '|'. Esta expresión única es, en la práctica, la representación compacta del autómata finito completo que reconoce el lenguaje léxico de un Dockerfile.

Paso 5 - Implementar el bucle de lectura (tabla de transiciones)

La función analizar(codigo_fuente) recorre el texto de entrada con TOKEN_REGEX.finditer(), lo que emula la lectura carácter a carácter descrita en la teoría: por cada coincidencia se determina el tipo de token (match.lastgroup), se descartan los tokens que deben ignorarse (SKIP, COMMENT, NEWLINE) y se contabiliza la posición (línea y columna) para fines de reporte.

Paso 6 - Definir los estados fallidos y el reporte de errores

Siguiendo la recomendación teórica de no detener el análisis en el primer error, el patrón MISMATCH ('.', cualquier carácter) actúa como estado fallido: captura cualquier carácter no reconocido por ningún patrón anterior, lo registra en la lista de errores (lexema, línea, columna, motivo) y el autómata regresa automáticamente al estado inicial para continuar leyendo el resto del archivo. Se agregó además el token UNKNOWN_INSTRUCTION para detectar palabras en mayúsculas al inicio de línea que no correspondan a ninguna instrucción Docker válida (por ejemplo, FORM en lugar de FROM), replicando el ejemplo de la lectura sobre identificadores no reconocidos (wile en vez de while).

Paso 7 - Construir la interfaz de línea de comandos

La función `main()` permite ejecutar el lexer sobre un archivo pasado como argumento (`python3 lexer_dockerfile.py archivo`) o leyendo desde la entrada estándar, e imprime cada token reconocido junto con su lexema, línea y columna, así como un resumen final indicando si el archivo es léxicamente válido o si contiene errores. El código de salida del proceso es 0 (válido) o 1 (con errores), útil para integrarlo en un pipeline de verificación (CI/CD).

Paso 8 - Validar el lexer con casos de prueba

Finalmente se diseñaron tres Dockerfiles de prueba —dos válidos (uno simple y uno multi-stage) y uno con errores léxicos intencionales— para verificar que el lexer clasifica correctamente los tokens y detecta los errores esperados.

Manual de Usuario del Metacompilador (Flex) .

El Flex es una herramienta útil que ayuda a crear analizadores léxicos de forma automática esto es importante cuando se desarrollan compiladores y procesadores de lenguajes. La función principal del Flex es tomar una especificación de patrones, que se definen usando expresiones regulares, y convertirla en un programa que funciona y está escrito en el lenguaje de programación C.

Estructura del Archivo de Especificación (.l)

El funcionamiento de Flex se basa en un archivo de entrada con extensión `.l`, el cual se divide obligatoriamente en tres secciones separadas por el delimitador `%%`:

- a. **Sección de Definiciones (`%{ ... %}`):** Es la parte inicial del archivo. Se utiliza para declarar librerías de C necesarias (por ejemplo, `#include <stdio.h>`), configurar opciones del compilador y definir variables globales o constantes que el lexer utilizará durante su ejecución.
- b. **Sección de Reglas (`%% ... %%`):** Esta es la sección central y más importante. Aquí se definen los patrones léxicos utilizando expresiones regulares y se les asigna una acción asociada, que es el código C que se ejecutará cuando el analizador encuentre una coincidencia en el texto de entrada.

Es en esta parte donde se realiza la **tokenización**, permitiendo clasificar elementos como identificadores, palabras reservadas y operadores.

- c. **Sección de Código de Usuario:** Es la última parte del archivo, donde se incluye la lógica principal del programa. Generalmente, aquí se define la función `main()`, la cual se encarga de abrir el archivo fuente que se va a analizar y de invocar a la función `yylex()`, la cual inicia el proceso de escaneo y clasificación de tokens.

El uso de Flex tiene varias ventajas.

La primera ventaja es que es muy eficiente esto se debe a que Flex automatiza la creación de un Autómata Finito Determinístico, lo que facilita el reconocimiento de lexemas, otra ventaja importante es la facilidad para realizar cambios si se necesita agregar un nuevo componente léxico, solo hay que definir una nueva regla en la sección correspondiente esto hace que sea mucho más fácil mantener y actualizar el lenguaje. Además, Flex facilita la gestión de errores se debe a que permite implementar reglas para capturar caracteres no reconocidos de manera sencilla es fundamental para informar al usuario sobre errores léxicos en el programa fuente.

En resumen, el uso de Flex ofrece ventajas significativas en términos de eficiencia, mantenibilidad y gestión de errores.

Descripción del Lenguaje *L*.

El lenguaje *L* se creó como una versión simplificada del lenguaje de programación Rust. Su objetivo es demostrar que se puede automatizar el análisis léxico con metacompiladores este lenguaje puede reconocer estructuras básicas como declaraciones de variables y operaciones aritméticas.

| Columna1 | Columna2 | Columna3 |
|----------|-----------------|--|
| Token | Lexema / Patrón | Descripción |
| LET | let | Palabra reservada para declarar variables |
| MUT | mut | Palabra reservada para indicar mutabilidad |
| FN | fn | Palabra reservada para definir funciones |

| | | |
|---------------|------------------------|--------------------------------------|
| TIPO_INT | i32 | Especificador de tipo de dato entero |
| BOOLEANO | true "false" | Valores lógicos |
| IDENTIFICADOR | [a-zA-Z_][a-zA-Z0-9_]* | Nombres de variables o funciones |
| NUMERO | [0-9]+ | Literales numéricos enteros |
| ASIGNACION | = | Operador de asignación |
| SUMA | + | Operador aritmético de suma |
| PUNTO_COMA | ; | Delimitador de fin de sentencia |
| LLAVE_ABRE | { | Delimitador de inicio de bloque |
| LLAVE_CIERRA | } | Delimitador de fin de bloque |

Características del Diseño

- **Gestión de Espacios:** El lenguaje ignora automáticamente espacios en blanco, tabulaciones y saltos de línea, los cuales son descartados durante la fase de análisis léxico.
- **Manejo de Errores:** Se ha implementado una regla comodín que identifica cualquier carácter no incluido en los tokens definidos anteriormente, clasificándolo como un "ERROR LÉXICO" para informar al usuario de inconsistencias en el código fuente.
- **Sintaxis:** El lenguaje mantiene una estructura sencilla que permite evaluar la correcta secuencia de palabras clave, identificadores y operadores, facilitando su posterior validación sintáctica.

Proceso de Implementación

Para crear y poner en marcha el analizador léxico, seguimos una serie de pasos en un entorno especial utilizamos WSL, que es como un sistema operativo Linux dentro de Windows. Esto nos permitió usar herramientas de compilación de GNU directamente en Windows.

Preparación del Entorno

Creamos un espacio de trabajo aislado con VS Code y la extensión WSL. Luego, configuramos este espacio instalando lo que necesitábamos con el gestor de paquetes de Ubuntu.

```
sudo apt update
```

```
sudo apt install flex gcc build-essential -y
```

Este comando nos permite asegurarnos de que tengamos el compilador de C (gcc) y la herramienta de metacompilación flex. De esta forma, podemos procesar el lenguaje.

Creación del Analizador

Creé un archivo llamado rust_lexer.l. Allí, integré las reglas del autómata en tres secciones:

- Declaraciones: Configuro el entorno de inclusión.
- Reglas: Defino los patrones que coinciden con los tokens del lenguaje. Estos patrones están hechos con expresiones regulares.
- Código de Usuario: Incluyo la lógica para leer y escribir datos. Esta lógica llama a la función yylex() para procesar el código fuente.

Generación y Compilación

Una vez lista la especificación, seguí estos pasos en la terminal bash para convertirla en un programa que se puede ejecutar:

Generar archivo C

```
flex rust_lexer.l
```

Este comando procesa las reglas. Como resultado, se crea el archivo lex.yy.c. Este archivo tiene el código fuente en C del autómata.

Compilar ejecutable

```
gcc lex.yy.c -o rust_lexer
```

Compilo el archivo lex.yy.c con gcc. De esta forma, obtengo el programa rust_lexer.

Ejecutar y Verificar

```
./rust_lexer prueba.rs
```

Ejecuto el programa con el archivo prueba.rs. De esta forma, puedo verificar si detecta correctamente cada token que define.


```

pc@DESKTOP-78TR84V:~/ProyectoLexer$ flex rust_lexer.l
pc@DESKTOP-78TR84V:~/ProyectoLexer$ gcc lex.yy.c -o rust_lexer
pc@DESKTOP-78TR84V:~/ProyectoLexer$ ./rust_lexer prueba.rs
TOKEN: LET, Lexema: let
TOKEN: MUT, Lexema: mut
TOKEN: IDENTIFICADOR, Lexema: x
TOKEN: ASIGNACION, Lexema: =
TOKEN: NUMERO, Lexema: 10
TOKEN: PUNTO_COMA, Lexema: ;
TOKEN: FN, Lexema: fn
TOKEN: IDENTIFICADOR, Lexema: test
TOKEN: LLAVE_ABRE, Lexema: {
TOKEN: LET, Lexema: let
TOKEN: IDENTIFICADOR, Lexema: y
TOKEN: ASIGNACION, Lexema: =
TOKEN: NUMERO, Lexema: 20
TOKEN: PUNTO_COMA, Lexema: ;
TOKEN: LET, Lexema: let
TOKEN: IDENTIFICADOR, Lexema: resultado
TOKEN: ASIGNACION, Lexema: =
TOKEN: IDENTIFICADOR, Lexema: x
TOKEN: SUMA, Lexema: +
TOKEN: IDENTIFICADOR, Lexema: y
TOKEN: PUNTO_COMA, Lexema: ;
TOKEN: LLAVE_CIERRA, Lexema: }

```

Indague y reflexione en que lenguajes en el área de seguridad informática, pudiera aplicar un FLEX, presente el lenguaje y su respectiva tokenización.

Aplicación de Flex en el Área de Seguridad Informática

En el área de la seguridad informática, es fundamental procesar y analizar grandes cantidades de datos en tiempo real. Flex es muy útil para crear herramientas que analicen el tráfico, detecten intrusiones y revisen los registros, lo que permite identificar patrones maliciosos dentro de un flujo de datos.

Lenguaje Seleccionado: Un subconjunto del lenguaje de reglas de Snort ya que es uno de los sistemas de detección de intrusiones más comunes. Utiliza un lenguaje de reglas para detectar ataques, como intentos de inyección de código o escaneos de puertos. Flex es ideal para dividir estas reglas en partes y verificar si son válidas antes de cargarlas en el sistema que inspecciona los paquetes.

Ejemplo de Tokenización

Imagina una regla simple de seguridad: alerta de conexión a través de tcp desde cualquier lugar hacia la dirección 192.168.1.1 en el puerto 80, con el mensaje "Intento de acceso web".

Al aplicar una tokenización diseñada con Flex a este lenguaje de reglas, obtendremos la siguiente clasificación:

| Token | Lexema | Descripción |
|------------|-------------------------|-----------------------------------|
| ACCION | alert | Define qué hacer ante el evento |
| PROTOCOLO | tcp | Define el protocolo de transporte |
| DIRECCION | any | Indica origen o destino de red |
| FLECHA | -> | Operador de dirección del tráfico |
| IP | 192.168.1.1 | Dirección IP objetivo |
| PUERTO | 80 | Puerto destino |
| MSJ_INICIO | (msg: | Inicio de metadatos de alerta |
| CADENA | "Intento de acceso web" | Descripción de la firma de ataque |

Reflexión Técnica

La implementación de un lexer basado en Flex para herramientas de seguridad brinda beneficios importantes:

- Alta velocidad: Convierte reglas de texto simple en tokens. Así, el motor de seguridad puede avanzar rápido hasta comparar firmas. Esto reduce la demora en la red.
- Validación de Políticas: Verifica que las reglas de seguridad que un administrador ha ingresado tengan la sintaxis correcta antes de implementarlas. Evita que se bloquee tráfico legítimo por errores de escritura.
- Filtrado de Logs: En ciberseguridad, un lexer puede procesar archivos syslog de gigabytes. Sirve para detectar intentos fallidos de autenticación (p. ej., varios errores de "Contraseña fallida") solo mediante la tokenización de eventos críticos.

Conclusión

El análisis detallado y la implementación práctica realizados a lo largo de este trabajo permiten consolidar una visión clara sobre el rol crítico que desempeñan los autómatas y los analizadores léxicos en la ingeniería de software actual. En primer lugar, la simulación del Autómata de Pila para el lenguaje de cadenas balanceadas y la validación de paréntesis demuestran que la incorporación de una estructura de memoria interna es un requisito indispensable para dotar a los sistemas de capacidades recursivas. Sin este salto conceptual, tareas cotidianas como la evaluación de expresiones matemáticas o la interpretación de formatos jerárquicos como HTML o JSON serían computacionalmente imposibles para los autómatas de estados finitos.

Por otro lado, la transición hacia la práctica mediante la construcción del analizador léxico para Dockerfile y el uso de Flex pone de manifiesto la versatilidad de los métodos de análisis. Mientras que el enfoque manual con expresiones regulares en Python ofrece una gran legibilidad y facilidad de integración para herramientas de automatización rápida, la automatización provista por Flex destaca por su rendimiento óptimo y velocidad de procesamiento a bajo nivel. La generación automática de un Autómata Finito Determinista en lenguaje C reduce significativamente los tiempos de desarrollo y mantenimiento, garantizando un manejo robusto de los errores léxicos.

Finalmente, la extrapolación de Flex al área de la seguridad informática, específicamente en la tokenización de reglas para Snort, subraya que los principios de la teoría de lenguajes formales no se limitan exclusivamente a la creación de compiladores tradicionales. En el contexto de la ciberseguridad, donde la velocidad y la precisión son críticas, la capacidad de procesar masivamente firmas de tráfico de red, registros de sistema y políticas de acceso mediante analizadores basados en autómatas es la clave para la detección temprana de amenazas y la mitigación de ataques en tiempo real. En conclusión, el estudio de los autómatas y los traductores no representa únicamente una base teórica histórica, sino una caja de herramientas viva y esencial para la optimización, el análisis y la protección del ecosistema tecnológico moderno.